

Argus × n8n

Automations, an inbound endpoint, and retiring the native connectors

Design proposal · 1 August 2026 · Project Argus · brainstormed with Ethan

1. Summary

Argus gains an **Automations** subsystem built on n8n. It works in both directions. Outbound, Argus discovers workflows you have tagged `argus` in your own n8n instance and renders each one as a card — a form derived from the workflow's own n8n Form Trigger, plus a button that fires it. Inbound, Argus exposes a small authenticated HTTP endpoint through a tunnel it manages itself, so workflows running anywhere (a laptop, a VPS, a phone-triggered flow) can capture notes and tasks into the vault and push structured data that Argus renders as live dashboard widgets.

The native Google Calendar and Todoist connectors are retired and replaced with shipped n8n workflow templates that install into your n8n with one click.

The four parts, in delivery order

#	PART	WHAT IT DELIVERS
1	Outbound automations	Register one n8n instance. Discover tagged workflows, render Form Trigger fields as native forms, fire them, and render what they return — an acknowledgement, a pass/fail status, or an inline widget. No tunnel, no inbound surface, no security exposure.
2	Inbound endpoint + tunnel	<code>/api/external/*</code> on its own port behind a bearer token, plus managed <code>cloudflared</code> . Capture and read only.
3	Widgets + ambient readout	Declared widget kinds rendered on the dashboard, with staleness detection surfaced in the HUD status line.
4	Templates + migration	Gallery of installable workflow templates; Google Calendar and Todoist move onto them; <code>backend/connectors/</code> is deleted.

WHY PART 4 IS LAST

Deleting working code is the final step, not the first. Parts 2 and 3 must be proven in daily use before five live surfaces — `TASKS.DUE`, `PLANNER.TIMELINE`, the 07:00 briefing, insights, and the agenda — are moved onto an unproven path.

2. Motivation, and one reversal worth recording

Three drivers: n8n owns credential management so Argus stops reimplementing it per service; n8n's node library reaches services Argus would never write first-party connectors for; and n8n is a safe place to put *write* actions, which today's read-only connectors cannot do at all.

Argus as it stands is thin on automation, and the goal is to scale it by adding your own — fire them from Argus, or read what they capture in Argus.

REVERSAL DURING DESIGN — KEEP THE REASONING

The initial recommendation was to *keep* the native gcal and todoist connectors and make n8n purely additive. The reasoning: routing them through n8n replaces "paste a token" with "stand up n8n, then configure credentials in it," which is strictly more setup, and it makes five working surfaces depend on a second service staying up.

Shipped workflow templates overturn that. Today's Google Calendar setup is: Cloud Console → new project → enable the Calendar API → create a Desktop OAuth client → download JSON → paste into Argus → browser consent. With a template it is: click *Google Calendar* in the gallery → Argus pushes the workflow into your n8n → click Connect on the Google node → done. That is genuinely less work, and it makes full replacement the right call rather than a costly one.

The win is uneven between the two services, though, and §2.1 states it precisely rather than claiming it broadly.

2.1 What actually changes about credentials

Google Calendar and Todoist do not benefit equally, and it is worth being exact.

SERVICE	TODAY (NATIVE)	N8N CLOUD	N8N SELF-HOSTED
Todoist	Settings → Integrations → Developer → copy token → paste into Argus. Two steps.	Node → <i>Connect my account</i> → OAuth. No token hunting.	The same token from the same place, pasted into n8n instead. A lateral move.
Google Calendar	Cloud Console → project → enable API → consent screen → Desktop OAuth client → download JSON → paste → consent. Seven steps , and testing-mode refresh tokens expire every 7 days.	One click — n8n ships a verified Google OAuth app. The 7-day expiry disappears with it.	Cloud Console once more (Web-application client, n8n's redirect URI). Comparable effort to today.

So the honest claim is: **Google Calendar gets substantially easier, Todoist does not.** Todoist's credential flow was never the problem.

Three things change regardless of which column you are in, and they matter more than the step counts:

- **It amortises.** One Google credential in n8n serves Calendar, Gmail, Drive, Sheets, Tasks and every Google workflow built later. Argus's model is one OAuth client per Argus, reusable for nothing.
- **Token refresh stops being Argus's code.** Expiry, refresh and re-auth become n8n's problem; `gcal.py` owns that today.
- **Argus stops holding third-party secrets entirely.** After migration its keyring holds one n8n API key and one external bearer token — no Google refresh tokens, no Todoist tokens. A real reduction in blast radius, and arguably the strongest argument for the migration.

What the template installer pre-fills: the callback URL, the bearer token, the schedule, the node wiring and the widget payload mapping. What it deliberately does not: your Google or Todoist credential. n8n's API can create credentials, but OAuth still needs a browser round-trip, and doing it for API-key types would mean handing Argus the secret again — defeating the third point above. Granting the credential in n8n is the one manual step per service, by design.

3. Decisions taken

QUESTION	DECISION	CONSEQUENCE
Where n8n runs	Anywhere — local or VPS	Argus knows a base URL and an API key and nothing else. No Node runtime in the installer.
Direction	Both; inbound is the headline	Argus must be reachable, authenticated, and must hold I3 at a network boundary.
Reachability	Argus manages its own tunnel	A publicly resolvable URL fronts an API that writes to the vault. Drives \$7 entirely.
Inbound surface	Capture/write + read structured data	No agent invocation, no action triggers. No LLM cost, no I6 obligation, no long requests.
UI source	n8n Form Trigger, via n8n's REST API	n8n stays the single source of truth for input schemas. Nothing configured twice.
Render contract	Payload declares its widget kind	Closed renderer set. Predictable, testable, keeps the Terminal-HUD identity.
gcal / todoist	Replaced entirely	<code>backend/connectors/</code> is deleted in part 4. Requires staleness detection to be real.
Notion reference	Views over captured data; dynamic rendering	A weather workflow pushes data and Argus draws it — not a JSON dump.
JARVIS reference	Ambient status readout	A persistent HUD line: what fired, what failed, what went quiet.

4. Architecture

4.1 One registered n8n instance

Base URL plus API key, key in the OS keyring (l4), verified with a live `GET /api/v1/workflows?limit=1` before anything is persisted. This is deliberately the same shape as the external MCP server registry in `backend/features/integrations/router.py` — register, probe, list, delete — because that is the one flow in the repo that already handles secrets correctly, including the ordering rule that a registry entry is written before its keyring secret so a crash leaves a visible, deletable row rather than an orphaned credential.

4.2 Module map

```
backend/features/automations/  
  n8n_client.py  httpx client for n8n's REST API — list, fetch, create, activate  
  schema.py      Form Trigger node -> typed field schema; widget payload validation  
  store.py       sqlite: registered automations, latest widget payloads, run history  
  catalog.py     the shipped template gallery  
  templates/     *.json — n8n workflow definitions bundled with Argus  
  router.py      /api/automations/* (local, unauthenticated, same as all /api)  
  
backend/features/external/  
  router.py      /api/external/* (public, bearer token)  
  auth.py        token generation, keyring storage, constant-time verification  
  app.py         a second FastAPI app mounting only the above  
  
backend/core/tunnel.py  
  cloudflared lifecycle: fetch, verify, spawn, supervise, report  
  
web/app/(dashboard)/automations/page.tsx  
web/components/automations/  
  AutomationCard.tsx  form + RUN button  
  WidgetRenderer.tsx  dispatch on widget kind  
  widgets/            one component per kind  
  TemplateGallery.tsx
```

`n8n_client.py` uses **httpx directly**, not a generated client — the same reasoning that governed the OpenAI-compatible adapter in the multi-agent work: no new PyInstaller surface in the frozen desktop backend, and `httpx.MockTransport` makes the whole thing testable with no live n8n.

4.3 The exposure boundary is a port, not a middleware check

THE SINGLE MOST IMPORTANT ARCHITECTURAL DECISION HERE

The external router runs on its *own uvicorn server, on its own localhost port*, and the tunnel points at that port. The main application on 8000 stays unauthenticated and stays unreachable from outside.

With a path-prefix check inside one app, a single routing mistake anywhere in Argus leaks the entire vault through a public URL. With a separate process boundary, a bug in the main app is simply not reachable from the internet. The safety property becomes structural rather than a thing that has to be remembered at every future endpoint.

5. Part 1 — Outbound automations

5.1 Discovery: tagging is the registration

Argus polls `GET /api/v1/workflows?tags=argus`. Tagging a workflow `argus` in n8n is the entire opt-in — nothing is registered a second time in Argus, and untagging removes the card. Registering an instance therefore does not dump forty unrelated internal workflows onto your dashboard.

5.2 Schema: the Form Trigger is the contract

For each discovered workflow, Argus fetches the full definition and inspects its trigger node.

TRIGGER NODE	RENDERED AS
<code>n8n-nodes-base.formTrigger</code>	A native Argus form. Each entry in <code>formFields.values[]</code> maps <code>fieldLabel</code> → label, <code>fieldType</code> → input type (text / number / date / dropdown / textarea / email / file), <code>requiredField</code> → required, <code>fieldOptions.values[]</code> → select options.
<code>n8n-nodes-base.webhook</code>	A bare button — no inputs.
Anything else (schedule, etc.)	Not shown as an action card. It may still own a widget via part 3.

Firing POSTs to the constructed URL — `{base}/form/{webhookId}` for a form trigger, `{base}/webhook/{path}` for a webhook. Rename a field in n8n and the Argus form changes on the next poll. There is no second schema to maintain.

5.3 Automations are also agent tools

A workflow with a typed field schema already is a `ToolSpec` in the sense the multi-agent adapter layer defines. Registered automations are therefore exposed to the Argus agent as callable tools at near-zero additional cost, so "run my meeting prep" works in the chat dock as well as by button. This is the JARVIS-shaped affordance, and it falls out of part 1 rather than needing its own part.

5.4 Run results — what happens after RUN

A HOLE IN THE FIRST DRAFT, NOT A REFINEMENT

Firing was originally specified as fire-and-forget: Argus POSTs to the webhook and that is the end of it. Any result only appears if the workflow separately pushes a widget, and only on the next refresh. So you press RUN and nothing visibly happens.

That is wrong for the entire class of automation you invoke *to get an answer* — "summarise the news on this topic", "look this up" — and wrong for plain actions too, where you at least want to know it worked.

Argus dispatches on what the workflow returns. One contract, three modes, no additional configuration: the response shape declares the mode, exactly as a widget payload declares its kind (§8.1).

WORKFLOW RETURNS	MODE	CARD SHOWS
nothing / immediate 200	ack	FIRED 12:04
<code>{"ok":true,"message":"Email sent to 3 recipients"}</code>	status	✓ with the message
<code>{"ok":false,"message":"Gmail: invalid recipient"}</code>	status	X with the reason
<code>{"widget":"text", ...}</code>	widget	The widget, rendered inline in the card

Timing. RUN is synchronous by default with a 30-second bound. A workflow that declares itself long-running (tag `argus:async`) instead gets a run id in its payload and pushes its result back to `POST /api/external/widget/{slug}?run={id}`; the card live-updates over the existing dashboard WebSocket. Multi-step LLM workflows routinely exceed any sane synchronous timeout, so this path is not optional.

N8N DEFAULT THAT PRODUCES A SILENTLY WRONG ANSWER

A Webhook node's *Respond* setting defaults to **Immediately**, which returns 200 *before* the workflow has done anything. Left alone, every action reports success instantly no matter what happens downstream. It must be set to *Using Respond to Webhook node* (or *When last node finishes*) for feedback to mean anything.

This belongs in the shipped template defaults and in the setup docs. It is a silently-wrong failure rather than a visible one, which makes it the most likely thing to go unnoticed in this whole feature.

When a workflow errors outright, n8n returns 500 with an error body and an execution id. Argus shows X with the message and a direct link to that execution in n8n, which is where it would actually be debugged.

5.5 Action safety

Actions differ from displays in two ways that need handling:

- **Confirmation on destructive actions.** A button that sends email or deletes something must not fire on a stray click. Tagging the workflow `argus:confirm` makes the card ask first — keeping configuration in n8n, one place, consistent with tagging being the registration.
- **In-flight locking.** The button disables while a run is open and each fire carries a run id, so a double-click cannot send two emails.

Run history matters more here than for widgets. A display leaves a persistent artefact to look at; an action leaves nothing, so "last 5 runs, 2 failed" on the card is the only trace there is.

6. Part 2 — The inbound endpoint

6.1 Surface

ENDPOINT	BEHAVIOUR
POST /api/external/capture	{title?, body, tags?, target?} → a note, written through <code>writer.py</code> . I1 holds unchanged.
POST /api/external/tasks	{text, due?, priority?, tags?} → a task, also via <code>writer.py</code> .
POST /api/external/widget/{slug}	Upsert a widget payload (part 3). First push creates the widget. An optional <code>?run={id}</code> routes the payload to a waiting card instead of the dashboard — the async half of §5.4.
GET /api/external/agenda?date=	Today's agenda, I3-filtered.
GET /api/external/tasks?status=open	Open tasks, I3-filtered.
GET /api/external/briefing?date=	A generated briefing, I3-filtered.
GET /api/external/ping	204, no body, no auth, no vault access. Tunnel liveness only.

Deliberately absent: agent invocation and action triggers. Neither was wanted, and their absence means no LLM spend behind a public URL, no I6 citation obligation on this surface, and no long-running requests holding a tunnel connection open.

THE BOUNDARY THIS DRAWS — WORTH KNOWING BEFORE IT IS HIT

Because agent invocation is excluded, **an n8n workflow cannot ask Argus a question grounded in the vault**. It can `GET /api/external/agenda` and feed the result to its own LLM node, but it cannot do retrieval over the notes.

So "summarise the news on X and show it in Argus" is in scope; "summarise *my notes* on X and post it to Slack" is not. The exclusion is defensible — it keeps model spend off a public URL — but it is the main thing this design cannot grow into without reopening this section.

6.2 Authentication and abuse resistance

- **Bearer token** — 32 random bytes, url-safe, generated by Argus, stored in the keyring under `argus-external/token`, compared in constant time. Rotation is a button; rotating invalidates the old token immediately and reprints the n8n credential.
- **Rate limit** — token bucket, 60 requests/minute, applied before any vault access.
- **Body cap** — 256 KB, rejected with 413. An unbounded public endpoint writing into sqlite and the vault is a disk-fill vector.
- **I3 at the boundary** — every read endpoint reuses the same filtered services the local API uses. `99-Private/` and anything tagged `#no-ai` must never appear in an `/api/external/*` response. This is the single most important test in the feature (§10).
- **No CORS, no cookies, no browser session**. This surface is for machine callers only.

6.3 One credential, configured once

Argus prints a ready-made n8n *Header Auth* credential — header name, token value, and your current tunnel URL. You paste it into n8n once; every subsequent custom workflow picks "Argus" from n8n's credential dropdown. No token copying per workflow.

7. The tunnel

`cloudflared` is fetched on first enable rather than bundled — it keeps the installer small — with checksum verification, into `.argus/bin/`. Argus spawns it as a supervised child, restarts it with exponential backoff on death, and reports UP / STARTING / DOWN plus the live URL to the HUD.

QUICK TUNNELS WILL SILENTLY BREAK EVERY WORKFLOW

A Cloudflare *quick tunnel* needs no account, but its `*.trycloudflare.com` hostname changes on every restart. Since that URL is baked into your n8n workflows, a restart silently breaks all of them — the workflows keep running and keep failing to reach anything.

So Argus supports both, and is explicit about the difference. Quick tunnel is offered for trying the feature out, labelled **EPHEMERAL — this URL changes when Argus restarts**. A named tunnel (you paste a Cloudflare tunnel token) is the supported configuration for real use, and is what the gallery templates assume. If the URL does change, Argus detects it and flags every installed template as needing its callback URL refreshed, rather than letting them fail quietly.

8. Part 3 — Widgets and the ambient readout

8.1 The declared-kind contract

A payload names its own renderer. Argus ships a closed set; unknown kinds are rejected at the endpoint with a message naming the valid ones, so a typo fails loudly at push time rather than rendering as an empty box hours later.

KIND	PAYLOAD SHAPE	USED BY
<code>metric</code>	<code>label, value, sub?, state?</code>	Weather, counters, single readings
<code>list</code>	<code>items[] {text, sub?, href?, state?}</code>	Todoist replacement
<code>table</code>	<code>columns[], rows[][]</code>	Arbitrary tabular captures
<code>timeline</code>	<code>entries[] {at, text, sub?}</code>	Google Calendar replacement
<code>text</code>	<code>body</code> (markdown, sanitised)	Summaries, digests
<code>chart</code>	<code>kind: line bar, series[] {label, points[] []}</code>	Trends over time

Every kind also accepts `title` and an optional `expected_interval_seconds`. A weather push looks like:

```
POST /api/external/widget/weather
Authorization: Bearer <token>

{ "widget": "metric",
  "title": "WEATHER",
  "label": "Manila",
  "value": "31°C",
  "sub": "feels like 38° · 70% humidity",
  "expected_interval_seconds": 1800 }
```

Widgets appear on `/automations` and can be pinned into the dashboard grid. Each renderer is a component under `web/components/automations/widgets/` built from the existing `ui/` primitives and the rem type scale, so pushed data cannot break the Terminal-HUD identity no matter what a workflow sends.

8.2 Staleness — why the readout is load-bearing

THIS IS WHAT MAKES REPLACING THE NATIVE CONNECTORS SAFE

The current connectors fail *at call time*: `list_tasks_safe` returns a message the UI can show. A push model fails *silently* — nobody calls anything, and the panel just shows yesterday's data or nothing at all. A silently-empty calendar is worse than a broken one.

So each widget row stores `last_seen_at` and an expected interval, either declared in the payload or inferred from observed push cadence. Past `2.5 × expected_interval` the widget takes on STALE chrome showing its true age, and the HUD status line names which source went quiet. Replacing gcal and todoist is defensible *because* this exists, not in spite of it.

The ambient readout is a persistent HUD line — the JARVIS-shaped part of this design. It reports the last automation fired, any run that failed, any source gone stale, and the tunnel's state, without you navigating

anywhere.

9. Part 4 — Template gallery and migration

Argus bundles n8n workflow definitions as JSON. Installing one `POST` s it to `/api/v1/workflows` with the `argus` tag applied and the callback URL and auth token already substituted into its HTTP Request node, then opens n8n at that workflow so you can grant the credential — the one step that genuinely must happen in n8n.

TEMPLATE	SHAPE	REPLACES
Google Calendar	Schedule → Google Calendar → POST <code>widget/calendar</code> as <code>timeline</code>	<code>backend/connectors/gcal.py</code>
Todoist	Schedule → Todoist → POST <code>widget/tasks</code> as <code>list</code>	<code>backend/connectors/todoist.py</code>
Weather	Schedule → weather API → POST <code>widget/weather</code> as <code>metric</code>	— new capability
Mobile capture	Webhook → POST <code>/api/external/capture</code>	— new capability

9.1 Migration sequence

1. Consumers (`TASKS.DUE` , `PLANNER.TIMELINE` , briefing, insights, agenda) are refactored to read from the automations store behind the *existing* interface, with the native connectors still supplying it.
2. The templates ship. Both paths can feed the store; the n8n path takes precedence when a source is registered and fresh.
3. After the n8n path has held up in daily use, `backend/connectors/` , `tests/connectors/` , the two connector rows in the integrations hub, and the `google-auth-oauthlib` / `todoist-api-python` dependencies are all deleted.

Step 1 is what makes step 3 a deletion rather than a rewrite: the consumers never learn where their data came from.

10. Failure modes and error handling

FAILURE	BEHAVIOUR
n8n unreachable	Cards render from the last cached schema, marked DISCONNECTED. RUN is disabled with the reason. The dashboard never blocks on n8n.
n8n API key rejected	Instance marked <code>failing</code> with the reason, matching the existing <code>ConnectorInfo</code> vocabulary. The key is not discarded.
Workflow fires but errors	n8n's execution id and error are captured in run history and surfaced on the card and the HUD line, with a link to that execution in n8n.
Workflow left on <i>Respond: Immediately</i>	Every run reports success regardless of outcome (§5.4). Argus cannot detect this from the response alone, so the mitigation is template defaults plus documentation — the one failure here that is invisible rather than loud.
Synchronous run exceeds 30s	The card reports TIMED OUT and links to the live execution in n8n. The workflow is not cancelled — n8n owns it. Tag it <code>argus:async</code> to move it onto the correlated path.
Async result never arrives	The waiting card falls back to its last known state after the run's TTL and records the run as unresolved in history.
Double-click on an action	Prevented — the button locks in flight and each fire carries a run id (§5.5).
A source stops pushing	STALE chrome with true age; named in the HUD readout. Never silently empty.
Malformed widget payload	422 naming the offending field. The previous good payload is retained and keeps rendering.
Unknown widget kind	422 listing valid kinds. Fails at push, not at render.
Tunnel dies	Supervised restart with backoff; HUD shows DOWN. Local Argus is entirely unaffected.
Tunnel URL changes	Detected; every installed template flagged as needing its callback refreshed.
Bad / rotated token	401, constant-time compare, no detail leaked. Repeated failures are rate-limited and reported in the HUD.
Write conflict on capture	Existing <code>writer.py</code> CAS and drift checks apply unchanged; 409 propagates to the caller.

11. Testing

- **The I3 boundary test is the critical one.** A note in `99-Private/` and a note tagged `#no-ai` must never appear in any `/api/external/*` response. Asserted per read endpoint, not once.
- `n8n_client` against `httpx.MockTransport` — the whole client is tested with no live n8n, matching the adapter precedent.
- Form Trigger → field schema, table-driven from real exported n8n workflow JSON, covering every `fieldType` and a workflow with no recognised trigger.
- Widget payload validation: valid and invalid cases per kind, plus unknown-kind rejection and the retain-last-good-payload behaviour.

- Run-result dispatch (§5.4): each of `ack`, `status` ok/fail, and `widget` resolves to the right card state, plus an n8n 500 producing `X` with the execution link.
- Synchronous timeout and the correlated async path — a run id issued, a later `?run={id}` push reaching the waiting card, and the TTL expiry when it never arrives. Injected clock, no sleeping tests.
- Action safety: `argus:confirm` gates the fire, and an in-flight lock makes a double-click produce exactly one run.
- Auth: absent, malformed, wrong, and rotated tokens; rate-limit and body-cap enforcement.
- Staleness with an injected clock — no sleeping tests.
- Tunnel lifecycle against a fake child process: spawn, unexpected death, backoff, URL-change detection.
- Playwright e2e against a stubbed n8n: install a template, fire a form automation, assert the widget renders, then advance the clock and assert it goes STALE.

Run with the venv interpreter — `.venv\Scripts\python.exe -m pytest` — or collection fails.

12. Open questions and spikes

1. **n8n public API shape.** Whether `webhookId` is reliably present on trigger nodes returned by `GET /api/v1/workflows` across n8n versions, and whether the tag filter behaves as documented. Blocks §5.2 URL construction.
2. **Community edition API availability.** Whether the public API is enabled by default on current self-hosted builds, or needs opting in. Changes the setup instructions.
3. **Named tunnel UX.** How much Cloudflare account setup Argus can absorb before "Argus opens the tunnel itself" stops being meaningfully easier than doing it yourself.
4. **Desktop packaging.** A second uvicorn port plus a downloaded binary interacting with the PyInstaller backend and the Electron shell — including Windows firewall prompts on first bind.
5. **Write-back scope.** Today's connectors are read-only. Whether replacement templates ever write *to* Google or Todoist, or stay read-only, is undecided and affects the credential scopes the templates request.

STANDING RISK

This feature puts a publicly resolvable URL in front of an API that writes to a personal Obsidian vault. The port boundary (§4.3), the bearer token (§6.2), and the I3 boundary test (§11) are the three things holding that line. None of them is optional, and none should be deferred to "later" during implementation.